

Language Model Forge: An Early-Stage Design Science Study to Simplify Language Model Customization

Yaman Aljnadi
Michigan Technological University

Sibonginkosi Nkashe
Michigan Technological University

Cletus Ngwerume
Michigan Technological University

Dr. Jeffrey D. Wall
Michigan Technological University
jdwall@mtu.edu

Abstract

Artificial intelligence (AI) models, such as language models (LMs), are growing in capabilities and utility. To further the study of domain-specific AI across various business domains, business researchers need access to simplified AI workflow tools to customize models. This study adopts a “pure technology” design science approach targeted at identifying the best automation configurations for a LM workflow tool, Language Model Forge (LMForge). LMForge utilizes simple linear interaction user interface (UI) views to hide complex AI workflow knowledge for common AI workflows into backend logic. This particular study focuses on a common AI workflow, creating custom fine-tuning datasets for later LM fine-tuning. Through two feasibility studies, we show that a two-model design with an encode-based transformer to identify relevant source data chunks and a decoder-based transformer model to generate question-answer pairs for a fine-tuning dataset is possible. Two studies are conducted to examine each model in isolation to determine the best of available foundation models. Based on the results of the studies, the configurations for the best encoder and decoder models are presented. With these models and configurations in place, future design science studies can experiment with the use of LMForge in context.

Introduction

Language models (LMs) and other forms of artificial intelligence (AI) are demonstrating great potential for use in organizations for automation and decision support (Elhajjar, Yacoub, & Yaacoub, 2023). Fully realizing this potential with the assistance of business faculty will require business researchers and students to study and be well-trained on the use of AI. The ability to create fine-tuning datasets, to use retrieval augmented generation (RAG) databases, and to fine-tune custom models and agents are quickly becoming differentiating skills for researchers across disciplines (Khalifa & Albadawy, 2024; Ligo et al., 2021). Although some business researchers and other social scientists possess the programming and data science workflow knowledge to create custom LMs and AI systems for domain-specific AI research, many do not (Ligo et al., 2021). Although this situation has prompted some collaboration between computing and social sciences faculty, domain-specific AI research is emergent and somewhat stunted (Ligo et al., 2021; Mariani, Machado, Magrelli, & Dwivedi, 2023).

The recent advent of graphical AI workflow tools like Langflow, n8n, and ChainForge (Arawjo, Swoopes, Vaithilingam, Wattenberg, & Glassman, 2024) provide support for those who lack strong programming skills. These graphical tools were built with the flexibility to support a multitude of AI use cases and user design creativity. However, Langflow, n8n, and ChainForge possess far more configuration options and tool connections than the average business researcher needs to study domain-specific LMs. These existing graphical workflow tools require users to design their own AI workflows, which may be difficult for those who don't understand foundations of AI-focused data science. While it is possible to train business and other social science researchers to use advanced graphical workflow tools, it is also possible to develop simpler workflow tools to support the creation of custom AI models for domain-specific research studies.

Many business research questions related to the use of AI in organizations are currently unstudied or poorly studied, yet represent important directions for future research (Mariani, Machado, Magrelli, & Dwivedi, 2023). To study the organizational effects of AI adoption and use, particularly in controlled experimental settings, better AI-customization support for a wider range of business researchers is needed. With better support tools, business researchers can work to answer important questions about the use, adoption, and behavioral and organization outcomes of AI use in organizations. For example, with the ability to customize LMs quickly and simply, business researchers could explore how domain-specific LMs affect individual, team, and organizational behavior through experimentation and case studies. With tools that provide simpler AI artefact generation, business researchers can also use their specialized business domain expertise to engage in design science research to identify new types of AI automations and decision supports for specific business problem domains (e.g., marketing, supply chain management, accounting, finance, etc.). Business researchers need simple tools that allow them to utilize their specialized domain knowledge to answer important AI-related domain questions that few others could answer.

This paper represents an early-stage study in a multi-study design science methodological effort to examine the design and use of simplified AI customization tools. Based on the current state of LM workflow applications, we propose the following problem statement for our design science study.

Problem Statement: Many business researchers do not possess the programming knowledge nor the data science workflow knowledge to quickly and effectively transition their specialized domain knowledge to study AI-related business questions within their specialized domains.

Design science is a scientific approach that integrates design and the application of scientific methods to improve the design and outcomes of IT artefacts for some domain context (Wieringa, 2014). Design science does not follow a single homogenous approach, but offers scientists a variety of methodological approaches to study problems, solutions, and the use of solutions in context (Venable, Pries-Heje, & Baskerville, 2016; Wieringa, 2014). While some design science studies examine the use of systems in context, other early-stage design science studies adopt a “pure technology” approach (Venable, Pries-Heje, & Baskerville, 2016). These “pure technology” design science studies rely on periodic formative evaluations or technical feasibility experiments to decide upon specific technological implementations for an IT artefact. Given the early stage of development of LMForge, this study employs a “pure technology” design science approach.

Design science research also relies on kernel design theories. Kernel design theories are not necessarily tested in design science studies, but instead, guide design efforts and ground design in good theories from domains such as management, psychology, sociology, etc. We adopt cognitive load theory, bounded rationality theory, and the technology acceptance model to inform our design decisions.

The remainder of this paper will continue as follows. First, a review of relevant literature is provided. Second, the kernel design theories are explicated. Third, the study methodology for the design science technical feasibility study is described. Fourth, the results of the feasibility study are provided. Last, a discussion of the study with directions for future research is offered.

Literature Review

Designing AI workflows has become increasingly simpler with the continued development of new programming libraries and software tools. For example, the tensorflow and torch libraries provide common tools for working with LMs. Similarly, HuggingFace provides a simple interface and API to manage repositories of LMs and datasets. HuggingFace has also published a number of useful programming libraries, such as transformers and SFTTrainer, that allow researchers and software developers to interact with models and datasets on HuggingFace to run and fine-tune models. These libraries are important to practitioners and researchers alike. However, their effective use often requires programming and data science knowledge.

A number of business focused LMs have been generated using libraries like these. For example, the FinGPT project offers a collection of language models that provide useful financial supports, such as sentiment analysis of financial news and fine-grained question answering to extract details from financial reports (Cao, Jiang, Wang, Yang, 2024; Danielsson, Macrae, Uthemann, 2022; Liu, Wang, Yang, & Zha, 2023). LM research in accounting is also growing. In the accounting domain, LMs summarize complex financial documents, identify inconsistencies, and assess risk, making them valuable tools for accountants and auditors seeking to improve decision-making and to detect irregularities in corporate reporting. One of the primary applications of LMs in accounting is their ability to summarize financial reports efficiently and assess risk (Yang, Zang, Ren, Peng, & Wen, 2024; Kim, Muhn, Nikolaev, & Tan, 2024). Similar studies can be found for other business domains as well.

Despite these and other studies, domain-specific business AI research is emergent and limited. To further enhance and support further research into the use of business AI research, we seek to simplify access to LM workflows by creating an interface that does not require the user to understand nuances in data science workflows. Instead, the user simply needs to know of domain-specific data sources that could be utilized in LM workflows. The interface guides the user through standardized workflows via linear interaction views to create fine-tuning datasets and to fine-tune language models. The long-term design science process of improving and adding new features to LMForge will both generate knowledge about ways to simplify AI workflows for non-technical users and will produce an IT artefact for continued study and use. Further, the continued integration of open-source projects, like LMForge, into interdisciplinary classes will provide space for the study of interdisciplinary open-source projects for educational purposes (Wall, 2023).

Kernel Design Theories

This study examines technical design choices for an important issue in the design of simplified AI workflow tools, namely how to abstract away design science workflow knowledge to grant non-technical users the ability to better utilize AI capabilities.

Although LMForge currently possesses multiple AI workflows, with more planned, fine-tuning an LM with a custom dataset to create a custom model is an important and common AI workflow that could be useful to non-technical business researchers. This workflow would allow non-technical business researchers to study domain-specific models in context and speed up knowledge creation about AI design and use across business domains. This particular workflow requires the user or system to: collect data, convert the data, chunk the data, remove irrelevant chunks (e.g., book preface, table of contents, index, etc.), generate fine-tuning data from the chunks, format the data into a specific fine-tuning template (often JavaScript Object Notation (JSON)), and then use the custom fine-tuning dataset to alter the model weights of a foundation AI model to create a custom model. Manually, this process would be time-consuming and near impossible for a business researcher with minimal knowledge of programming and AI workflows. We propose a system design that hides the programming and AI workflow design decisions

from the user by embedding the workflow details into a guided linear interaction through user interface views that have simple buttons and forms.

This design is informed by several kernel theories. First, we draw on theories of limited cognitive and rational capacity, namely cognitive load theory (CLT) (Sweller, 1988; Sweller et al., 1998) and bounded rationality theory (BRT) (Simon, 1955). Second, we draw on the technology acceptance model (TAM) (Davis, 1989) to enhance the utility of the design. We describe each theory briefly.

Cognitive Load Theory (CLT)

Cognitive load theory (CLT) suggests that human working memory has limited capacity. When cognitive demands go beyond this capacity, learning and task performance decline (Sweller, 1988). CLT identifies three types of cognitive load: *intrinsic cognitive load*, which comes from the task's inherent complexity; *extraneous cognitive load*, which arises from how a task or system is designed; and *germane cognitive load*, which represents the effort put into learning and building understanding (Sweller, Ayres, & Kalyuga, 1998).

We suggest that designing AI workflows (i.e., intrinsic load) is itself too complex a task for many non-technical users. For this reason, we seek to embed common workflows into simple linear interaction views, using AI and backend logic to hide the intrinsic complexities of designing a custom fine-tuning dataset. Tools like n8n and LangFlow reduce extraneous cognitive load for users by removing the need to know programming. However, n8n and LangFlow also possess a variety of tools in their toolsets, each of which can be configured individually and configured together through custom workflows. This design adds intrinsic and extraneous cognitive load that requires AI workflow knowledge, which many may not possess, and the selection of tools from a vast toolset that add to extraneous load. By restricting AI workflows to common workflows built into linear interaction views, LMForge reduces intrinsic, extraneous, and germane load.

By matching system demands with users' cognitive abilities, LMForge will allow non-technical business researchers to carry out AI customization tasks simply, effectively, and reliably. From a design perspective, CLT forms a solid basis for LMForge's focus on guided workflows, limited toolsets, and embedded and hidden expertise. These design choices are crucial for making AI adoption easier, while maintaining the technical quality needed for valid research and practical use.

Bounded Rationality Theory (BRT)

Bounded rationality theory (BRT) argues that human decision-making is fundamentally constrained by limited cognitive resources, incomplete information, and time pressures, such that individuals are unable to evaluate all possible alternatives or optimize complex decisions (Simon, 1955). Rather than engaging in fully rational optimization, individuals rely on simplified decision strategies, heuristics, and satisfactory behavior seeking solutions that are “good enough” to meet their goals under existing constraints (i.e., satisficing). This perspective

challenges assumptions of unbounded rationality that are often implicitly embedded in the design of complex technical systems.

From a bounded rationality perspective, system designs that require users to evaluate numerous options, compare multidimensional trade-offs, or reason about uncertain outcomes impose unrealistic expectations on human decision-making. Such systems implicitly assume that users possess sufficient expertise, time, and cognitive capacity to identify optimal solutions. In practice, these assumptions rarely hold, particularly for non-expert users operating in unfamiliar technical domains. As task complexity increases, the cognitive cost of decision-making rises sharply, increasing the likelihood of error, abandonment, or reliance on arbitrary choices. While tools like n8n and LangFlow grant intricate customization through custom workflow design, this complexity permits multiple designs to exist to accomplish the same core task. For a non-technical business researcher, these multiple design options could lead to overly simple workflow designs that fail to create the desired custom model.

By embedding common workflows with best practices into linear interactions, the user is required to make less decisions and thereby less opportunity for suboptimal satisficing behavior. The feasibility studies in this paper are designed to identify optimal configurations for the embedded workflow to ensure it is both simple and of high quality. In combination with CLT, BRT further clarifies the rationale for abstracting AI workflow complexity from LMForge. While CLT explains why excessive technical detail overwhelms users' working memory, BRT explains why non-technical users cannot reasonably be expected to make optimal technical decisions even when the right tools are made available. Together, these theories support a design approach in which complex AI workflows are transformed into guided, reliable processes that align with human cognitive and decision-making constraints.

Technology Acceptance Model (TAM)

The technology acceptance model (TAM) assumes that an individual's intention to use a technology is primarily determined by two core beliefs: *perceived usefulness*, defined as the degree to which a person believes that using a system will enhance task performance; and *perceived ease of use*, defined as the degree to which a person believes that using the system will be free of effort (Davis, 1989). TAM further proposes that perceived ease of use has both a direct effect on usage intention and an indirect effect through perceived usefulness, as systems that require less effort are more likely to be viewed as beneficial.

As a kernel theory for AI workflow tools, TAM would suggest that the tool should be able to handle useful workflows and produce the desired results of the workflow. For this reason, LMForge contains common AI workflows, such as the generation of fine-tuning datasets. TAM would also suggest a design that is easy to use. Linear interaction views (i.e., wizards) are common UI interfaces that are well understood by system users.

From a design science perspective, TAM provides a theoretical lens for understanding how abstracting details from common AI workflows into simple linear interaction UI views should influence design decisions for positive adoption outcomes. While CLT and BRT explain how

LMForge reduces mental and decision effort during task execution, TAM explains why these design choices are necessary for user acceptance. Without sufficient perceived ease of use and perceived usefulness, non-technical business researchers are unlikely to adopt AI workflow tools regardless of their technical merits.

A Two Model Design for Fine-Tuning Dataset Generation

For this particular design science study, we examine the design for generating fine-tuning datasets. Early trials conducted by the authors showed that a single off-the-shelf foundation LM was too prone to error when attempting to accept textual chunks of raw source data, filter out irrelevant chunks, and generate and format the data chunks into question-answer datasets. The design was constrained to be small enough to work on hardware affordable by a small research lab. This design constraint may explain why a single model could not do everything, as the constraint required the use of LMs in the range of 4-7 billion parameters. The complexity of handling relevance checking, question-answer generation, and formatting the output into JavaScript Object Notation (JSON) proved too much for these smaller LMs. It might be that larger labs could utilize a single larger model, but that use case lies outside of the design constraints of this study.

To accommodate smaller hardware profiles, and therefore make the tool more useful to a broader audience, we designed the system to utilize two distinct, yet complementary models. The first model is an encoder-based transformer model that we trained to classify data source chunks into relevant or irrelevant chunks. This model accepts the data source chunks and looks for common patterns, such as table of contents, indices, prefaces, etc., and then predicts whether the textual chunk is relevant or irrelevant. The system then filters out irrelevant chunks based on the classification of the encoder model and passes relevant textual chunks to an independent decoder-based transformer model. The second decoder model takes the relevant textual chunks and generates one to two question-answer pairs (depending on model instructions) from the chunks and formats the question-answer pairs into the correct JSON format. Through our experimentation described in the Methodology and Results sections, we identified a configuration for the appropriate textual chunk size, model output size, and specific encoder and decoder models to make this work efficient, with minimal failures. The configurations will be further tested within LMForge in context in future studies.

These two complementary models are hidden from the user, embedded in the backend logic. This design helps to add intelligent decisions in dataset creation while minimizing cognitive load. These models help to hide the details of the AI workflow steps, which aligns with CLT, RBT, and TAM. Appendix A shows screenshots from the LMForge system related to the models studied in this paper. The images are in the appendix because they are not core to this early-stage design science study.

Methodology

This study adopts a design science research methodology, specifically utilizing a "pure technology" approach. This approach is appropriate for early-stage design artifacts where the primary focus is establishing technical feasibility and optimal configurations before evaluating the system in a naturalistic organizational context (Venable et al. 2016).

Our larger multi-study design science approach will seek to understand how to design AI workflow tools for non-technical users to open AI research and practice to more people. This early-stage study examines the feasibility of the two-model dataset generation design described earlier. The study consists of two parts:

- An analysis of different encoder models to validate that textual data chunks are relevant (e.g., not preface text, index text, table of contents text, etc.).
- An analysis of different decoder models to generate JSON formatted fine-tuning datasets from relevant source data.

The research team started the decoder model study ahead of the encoder model study, but both studies were worked on in parallel. As such, the data for the decoder model was not passed through the encoder model to remove irrelevant text chunks. Instead, the data for the decoder model was cleaned programmatically for irrelevant text chunks by one of the authors.

Hardware Configuration for the Studies

The experiments were conducted on a laboratory workstation to demonstrate feasibility on accessible small lab hardware. The system specifications are as follows:

- Workstation: Dell Precision 7960 Tower
- Processor: Intel Xeon w7-3565X - 2.5 GHz (32 cores)
- GPU: NVIDIA RTX 5000 Ada Generation - 32 GB GDDR6 Discrete VRAM
- Memory: 132GB RAM - 4800 MT/s
- Operating System: Windows 11

Study 1: Encoder-Based Relevance Classification

To support the dataset generation workflow in LMForge, the design calls for an auxiliary encoder-based classification model focused on identifying relevant text chunks prior to downstream processing to generate fine-tuning datasets. This first step is critical for ensuring that only semantically meaningful content is passed into subsequent language model workflows, reducing noise and unnecessary computation. The first study examined the qualities of several encoder models.

Model Selection and Grouping

We evaluated a broad set of encoder-based transformer models commonly used for text classification tasks. These models were selected to represent a range of architectural sizes and computational profiles, allowing us to study performance–efficiency trade-offs relevant to

non-enterprise hardware environments. Importantly, even the large models are small enough to run on affordable hardware for small research labs. A total of 20 encoder models were examined for this study.

For clarity and comparability, models were grouped into three size-based categories:

- Small models (e.g., ALBERT-base variants, ELECTRA-small, MobileBERT): Designed for efficiency and low resource usage, suitable for rapid iteration and deployment on constrained hardware. In this study, small models are defined as architectures with fewer than approximately 50 million parameters.
- Medium models (e.g., DistilBERT, DeBERTa-v3-small, BERT-base-class models): Represent a balance between representational capacity and computational cost, often considered standard baselines for classification tasks. Medium models are defined as those with approximately 50 to 200 million parameters.
- Large models (e.g., RoBERTa-large, XLM-RoBERTa, DeBERTa-large variants): Provide higher expressive capacity at the cost of increased training and inference time. Large models are defined as architectures with more than approximately 200 million parameters.

Each model was fine-tuned for sequence classification using a consistent training pipeline implemented with the HuggingFace transformers library.

Experimental Groups

All models were evaluated under identical experimental conditions to ensure fair comparison. Text chunking was accomplished with the LangChain library. Input text chunks were tokenized and truncated using three maximum sequence lengths: 128, 256, and 512 tokens. These chunk sizes allowed us to examine the impact of contextual window size, constrained by chunk size, on classification performance. Larger chunk sizes (e.g., 1024 tokens) were not included in this first study due to the computational time and computing resource availability. Future work will extend this analysis to larger chunk sizes as computational time and resources are available.

Data Processing and Model Training Procedure

The dataset used in this study was generated from open-source text documents obtained from Project Gutenberg, selected to represent long-form, unstructured text similar to documents that will be processed by the LMForge system. Rather than utilizing the full Project Gutenberg corpus, a curated subset of documents was selected to ensure tractable annotation while preserving diversity in document structure and content. This dataset was used to train the selected encoder models to classify textual chunks as relevant (i.e., text related to the core purpose of the document) or irrelevant (i.e., text unrelated to the core purpose of the document, such as table of contents, indices, prefaces, etc.).

Source texts were segmented into smaller units (i.e., chunks) to ensure compatibility with transformer-based models while preserving sufficient semantic context. Each chunk constitutes a single classification instance.

Text chunks were assigned relevance labels indicating whether they contained information suitable for downstream processing. Relevance labels were assigned through a manual annotation process conducted by one of the authors. Relevant chunks corresponded to substantive content aligned with the primary narrative or informational goal of the document, while irrelevant chunks included structural or non-core content. Labeled chunks were stored alongside their corresponding text in a SQLite database and subsequently exported for model training and evaluation. To improve data quality, text chunks with undefined labels and extremely short positive samples were removed. A minimum length threshold of 20 tokens was applied for samples. Token counts used for dataset characterization and filtering were computed using a single reference tokenizer (BERT WordPiece) to ensure consistency across preprocessing steps.

The final dataset used to fine-tune the models consisted of 6,184 labeled text chunks, totaling 594,711 tokens. The final dataset was split into training and test sets using stratified sampling to preserve class distributions. Class imbalance was addressed during training through weighted loss functions rather than additional resampling. The training dataset was then used to train the models to detect relevance. The test dataset was used to evaluate the qualities of each model.

Training was conducted using the HuggingFace Trainer framework with hyperparameters adjusted according to model size. Learning rates ranged from 5×10^{-5} for smaller models to 1×10^{-5} for the largest models, reflecting differences in model capacity and stability during fine-tuning. Models were trained for 2 to 4 epochs, with smaller models trained for more epochs and larger models for fewer epochs to manage computational cost. Batch sizes were similarly scaled by model size, ranging from 32 for small models to 1 for the extremely large models. Input sequence lengths of 128, 256, and 512 tokens were evaluated for each model configuration. A weight decay of 0.1 was applied consistently for regularization across all experiments.

Evaluation Metrics

Model performance was assessed using standard classification metrics:

- Accuracy: the percentage of test samples for which the predicted label matched the ground-truth label.
- Weighted F1 score: emphasizing balanced performance across imbalanced classes.
- Precision and Recall: providing additional insight into error behavior.
- Training time and inference time: recorded to quantify computational efficiency.

Weighted F1 was used as the primary model selection criterion, as it provides a more informative measure than accuracy in the presence of class imbalance.

Study 2: Question-Answer Dataset Generation

The second study evaluated the generation of a question-answer fine-tuning dataset in JSON format that could be used for fine-tuning a model. To ensure the model provides a reliable experience for non-technical users, we conducted a rigorous comparative evaluation of potential

decoder-based language models. We used off-the-shelf foundation models for this study with few-shot learning examples showing the desired JSON output format. The objective was to identify a language model to balance speed (efficiency), strict adherence to formatting requirements (reliability), and semantic accuracy (quality).

Model Selection

We evaluated four prominent open-weight decoder language models suitable for affordable small research lab hardware:

- Microsoft Phi-4-mini-instruct
- Mistral-7B-Instruct-v0.3
- Qwen2.5-7B-Instruct
- Google Gemma-7B

These models were selected based on their prevalence in the 7-billion parameter class and for their acceptance as state of the art models at the time of commencing the study. The 7-billion parameter class of language models represents a sweet spot for local execution by researchers with limited computational resources.

Data Processing Procedure

The input corpus for the study consisted of two creative-commons business texts:

- *Legal Aspects of Corporate Management*
- *Finance and Principles of Finance.*

The textual data was processed using LangChain to split the documents into chunks of varying token sizes (C). To test the models' robustness under different constraints, we utilized a factorial experimental design. We defined four distinct chunk sizes ($C \in \{128, 256, 512, 1024\}$) and five maximum token generation limits ($T \in \{128, 256, 512, 1024, 2048\}$).

Each set of chunks was processed against every applicable token limit to observe how strict generation constraints affect model performance. This resulted in a total of 30,660 experimental trials per model. The total number of generation attempts (N) across all four models was calculated as follows:

$$N_{total} = M * \sum_{c \in C} (n_c * |T|)$$

Where M is the number of models, n_c is the number of text chunks for a specific size, and $|T|$ is the count of token limit configurations. Table 1 shows the configurations of the experiment by chunk size.

Table 1. Experimental Configurations for the Decoder Models by Chunk Size

Chunk Size	Number of Chunks	Model Max Tokens	Total samples
------------	------------------	------------------	---------------

128	1,800	128, 256, 512, 1024, 2048	9,000
256	1,800	128, 256, 512, 1024, 2048	9,000
512	1,622	128, 256, 512, 1024, 2048	8,110
1024	910	128, 256, 512, 1024, 2048	4,550

Each model was expected to output an appropriately formatted JSON response that included two question-answer pairs per textual input chunk. Code Example 1 shows an example of the expected format of the model output.

Code Example 1: Example of JSON output

```
{
  "chunk": "...Burger King sued in... federal district court... Rudzewicz contested the court's
personal jurisdiction...",
  "qa_pairs": [
    {
      "question": "What action did Burger King take?",
      "answer": "Burger King took legal action against Rudzewicz by suing in the federal district
court for the southern district of Florida."
    },
    {
      "question": "Why did Rudzewicz contest the court's personal jurisdiction over him?",
      "answer": "Rudzewicz contested the court's personal jurisdiction because..."
    }
  ]
},
...
```

Evaluation Metrics

The performance of the models was assessed using three primary dimensions crucial for generation workflows:

- **Efficiency:** Measured by the total elapsed time (in minutes) required to complete the generation tasks.
- **Reliability (Success Rate):** Defined as the percentage of model outputs that could be successfully parsed into valid JSON. This is a critical metric for this study, as non-technical users cannot be expected to debug syntax errors in generated datasets.
- **Quality:** Measured with two metrics:
 - *Semantic Accuracy:* Evaluated using BERT Score (F1) to measure how well the meaning of the generated Q&A preserved the context of the source text.
 - *Structural Quality:* Evaluated using ROUGE-L scores to measure phrasing overlap and structural adherence against reference standards.

Statistical Validation of Sampling Strategy

To ensure that the sampling approach, specifically the segmentation of text into varying chunk sizes, did not introduce harmful artifacts or inconsistent variability into the results, we conducted a multivariate dispersion analysis. For each of the four models, we standardized the

performance metrics (ROUGE, BERT Score, etc.) and computed the Euclidean distance of each sample to its group centroid, grouped by chunk size.

We then assessed the homogeneity of variances across the chunk size groups using both the Brown–Forsythe and Fligner–Killeen tests. These tests were selected to rigorously evaluate whether the variability in model performance differed significantly across chunk sizes, which would indicate instability in the sampling approach. The analysis confirmed that the sampling strategy was robust; all models exhibited high p-values ($p > 0.75$) for both tests, as shown in Table 2. The high p-value indicates no statistically significant difference in dispersion between chunk sizes, validating that the sampling method did not introduce bias.

Table 2. Homogeneity of Variance Tests for Data Sampling Strategy

Model	Brown-Forsythe (p-value)	Fligner-Killeen (p-value)
Google Gemma-7B	0.9630	0.7828
Microsoft Phi-4	0.9882	0.9919
Mistral-7B-Instruct-v0.3	0.9910	0.9476
Qwen2.5-7B-Instruct	0.7674	0.8371

Results

We now present the results of the two studies, starting with the analysis of the encoder-based transformer models and then transitioning to the decoder-based transformer models.

Study 1 Results: Encoder-Based Relevance Classification

To evaluate the effectiveness of encoder-based models for relevance classification, we benchmarked multiple transformer architectures across varying textual chunk input lengths. Results are reported using accuracy and weighted F1 score, with additional consideration for training and inference time.

Performance Details

Twenty different encoder models were tested with three different input chunk size configurations (i.e., 128, 256, and 512 tokens), leading to a total of 60 trained and tested models. Given the number of models tested, we do not provide a full report of each model. Instead, we provide a graphical heatmap of all the models, and tables of the best performing small, medium, and large models. Figure 1 shows a heatmap of all 60 models by chunk size length based on the weighted F1 evaluation metric. Figure 2 shows a heatmap of all of the models based on model accuracy scores. Figure 3 shows a heatmap based on the prediction time for each model in seconds. Reported prediction times are for all predictions for a dataset and not for individual data point predictions.

Figure 1: Heatmap of Weighted F1 Performance by Model

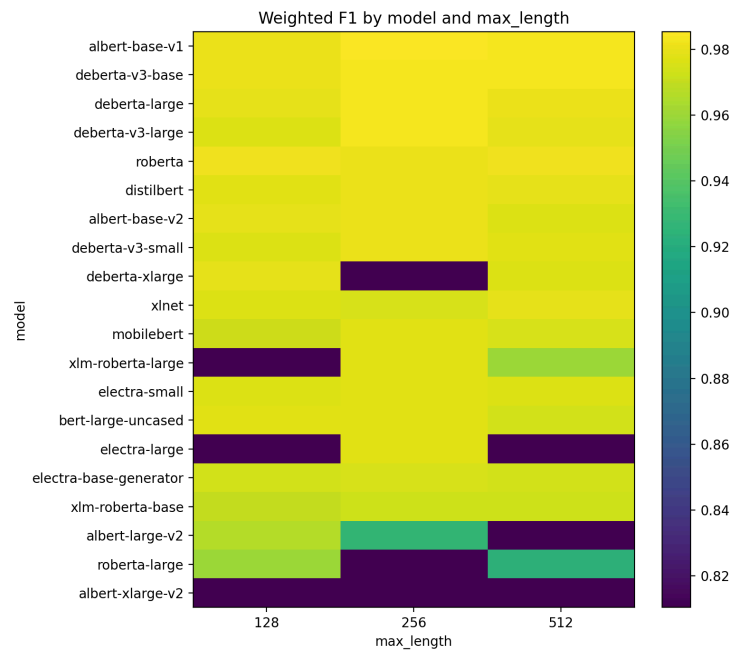


Figure 2. Heatmap of Classification Accuracy by Model

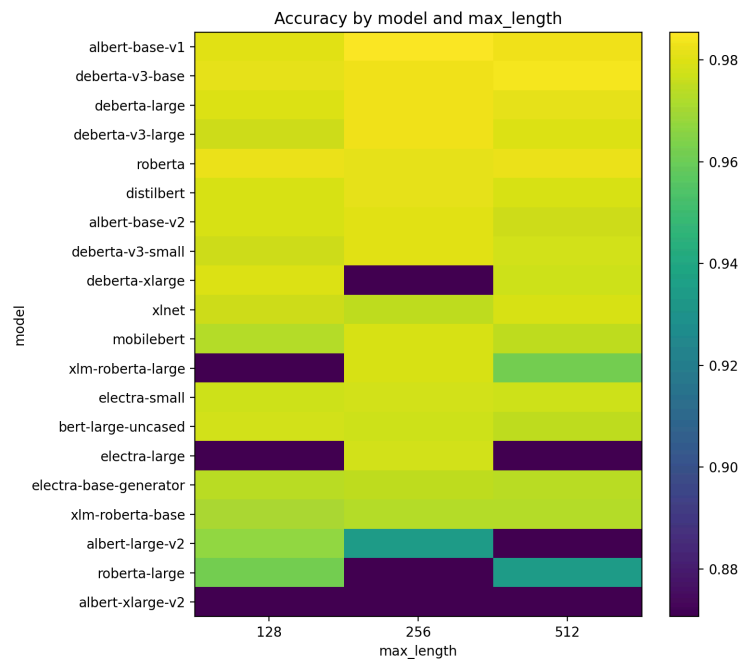
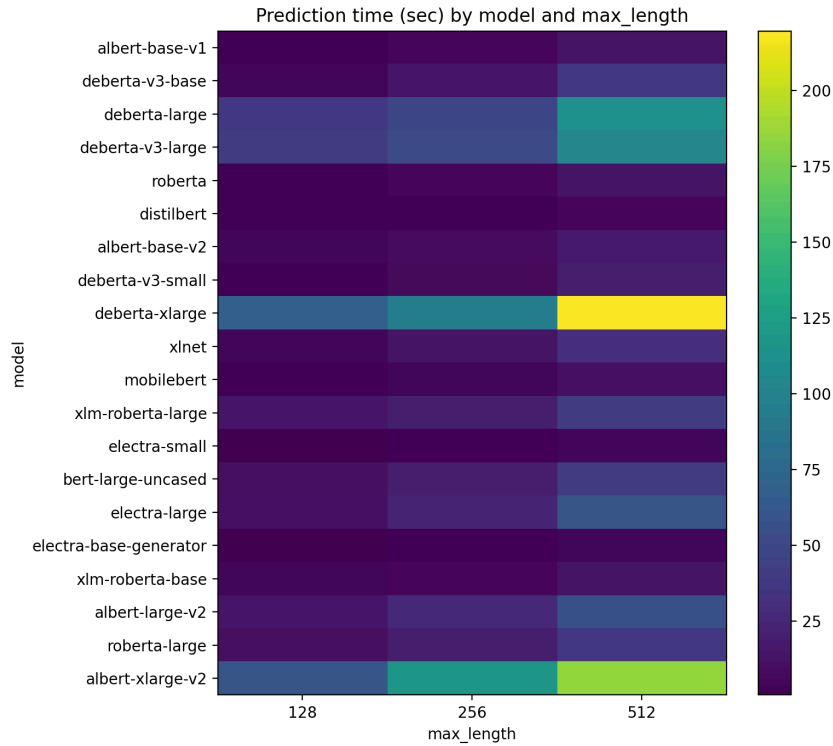


Figure 3. Heatmap of Model Prediction Time (in seconds)



The performance metrics of the top five best performing small encoder models is shown in Table 3. These small models showed acceptable accuracy scores (i.e., above 96%) with an average of 97.80% and an average Weighted F1 score of 0.9775, with inference times ranging from 5.86 to 13.77 seconds and an average time of 7.00 seconds.

Table 3. Best Performing Small Encoder Models

Model	Best Max Length	Accuracy	Weighted F1	Training Time (s)	Inference Time (s)
albert-base-v1	256	0.9854	0.9854	282.31	5.86
albert-base-v2	256	0.9806	0.9803	433.95	9.23
mobilebert	256	0.9790	0.9784	184.52	4.19
electra-small	256	0.9782	0.9779	98.24	1.96
albert-large-v2	128	0.9669	0.9656	649.74	13.77
Average		0.9780	0.9775	329.75	7.00

The performance metrics of the top five best performing medium encoder models is shown in Table 4. These medium models showed high accuracy scores (i.e., > 97%) with an average of

98.14% and an average Weighted F1 score of 0.9812, with inference times ranging from 3.23 seconds to 36.84 seconds and an average time of 17.96 seconds.

Table 4. Best Performing Medium Encoder Models

Model	Best Max Length	Accuracy	Weighted F1	Training Time (s)	Inference Time (s)
deberta-v3-base	512	0.9838	0.9837	1386.09	36.84
roberta	512	0.9822	0.9822	566.19	11.96
xlnet	512	0.9790	0.9786	1223.04	30.79
deberta-v3-small	256	0.9806	0.9802	305.15	6.99
distilbert	256	0.9814	0.9812	139.70	3.23
Average		0.9814	0.9812	724.03	17.96

The performance metrics of the top five best performing large encoder models is shown in Table 5. These large models also showed high accuracy scores (i.e., > 97%) with an average of 98.06% and an average Weighted F1 score of 0.9802, with inference times ranging from 10.58 seconds to 51.81 seconds and an average time of 40.28 seconds.

Table 5. Best Performing Large Encoder Models

Model	Best Max Length	Accuracy	Weighted F1	Training Time (s)	Inference Time (s)
deberta-large	256	0.9830	0.9827	2939.95	49.74
deberta-v3-large	256	0.9830	0.9827	3081.35	51.81
deberta-xlarge	128	0.9798	0.9793	4897.87	68.05
xlm-roberta-large	256	0.9790	0.9784	1774.17	21.24
bert-large-uncased	128	0.9782	0.9778	625.62	10.58
Average		0.9806	0.9802	2663.79	40.28

Overall Assessment of the Encoder Models

The results suggest that on average, the best medium models performed as well or better than the best large models, but did so on average in less than half the time. Given these results, the large models may not be the best choice for LMForge to operate efficiently in small research

labs. Similarly, the medium models performed only slightly better than the small models, with some models taking more inference time than might be desirable for use in LMForge. However, some of the medium models displayed high accuracy and weighted F1 scores and inference times below the average inference time of the small models. These faster inference medium models, namely distilbert deberta-v3-small, may be more advantageous for small research labs than some of the slower medium-sized models. One of the small models, albert-base-v1, also showed promising accuracy and weighted F1 scores above 98% that rival the medium and large models, with a fast interface time of 5.86 seconds.

Study 2 Results: Question-Answer Dataset Generation

To evaluate the decoder models, we conducted a large-scale feasibility study involving synthetic Q&A generation trials across four candidate models: Mistral-7B-Instruct-v0.3, Microsoft Phi-4, Qwen2.5-7B, and Google Gemma-7B. Each model processed the exact same set of 30,660 configuration inputs (as detailed in Table 1). For every generation trial, the model was prompted to produce two question-and-answer pairs derived from the text chunk. The performance of these models was assessed across three dimensions: efficiency (generation speed), reliability (valid output generation rate), and output quality (semantic and structural accuracy).

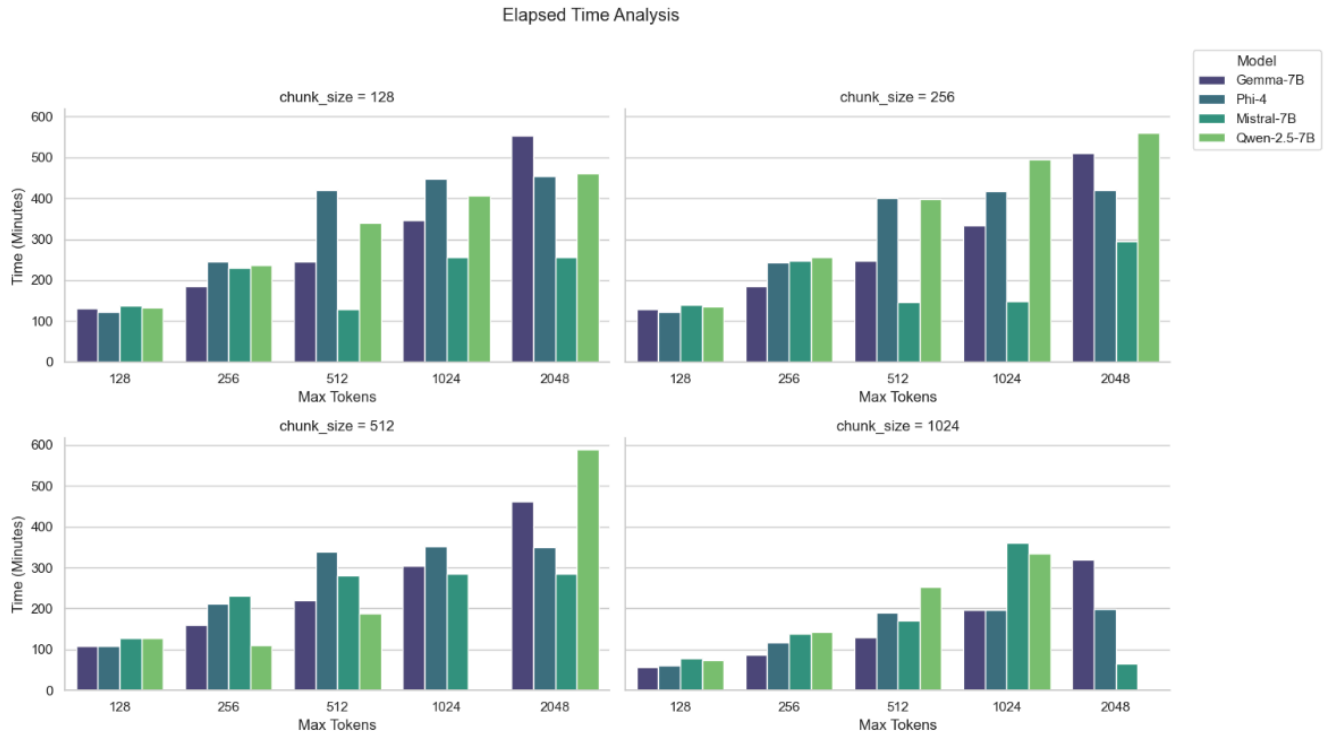
Efficiency Analysis

For the model to be viable in LMForge on small research lab hardware, the underlying model needed to balance generation speed with resource consumption. Our analysis tracked the total elapsed time required to generate datasets at varying chunk sizes and max output token sizes.

Figure 4 shows the average inference time in minutes to parse 900 data samples for the four decoder models, represented by different colors, with a chart for each chunk size and groups in each chart for each max output token size. As depicted in Figure 4, the general upward trends in inference times across the max output token sizes shows that allowing for the question-answer pair outputs to be more verbose may dramatically increase user wait times, with some clear exceptions.

The data also shows Mistral-7B was consistently at or below the inference times of the other models, particularly for larger max output token sizes. Mistral-7B averaged approximately 200 minutes per run across configurations. In contrast, Phi-4 averaged approximately 270 minutes, while Qwen2.5-7B and Gemma-7B required significantly more time. Comparative analysis indicates that Mistral-7B is roughly 25-30% more efficient in raw processing time than its nearest competitor, Phi-4, for this specific data task.

Figure 4: Comparison of Total Elapsed Generation Time (in minutes)

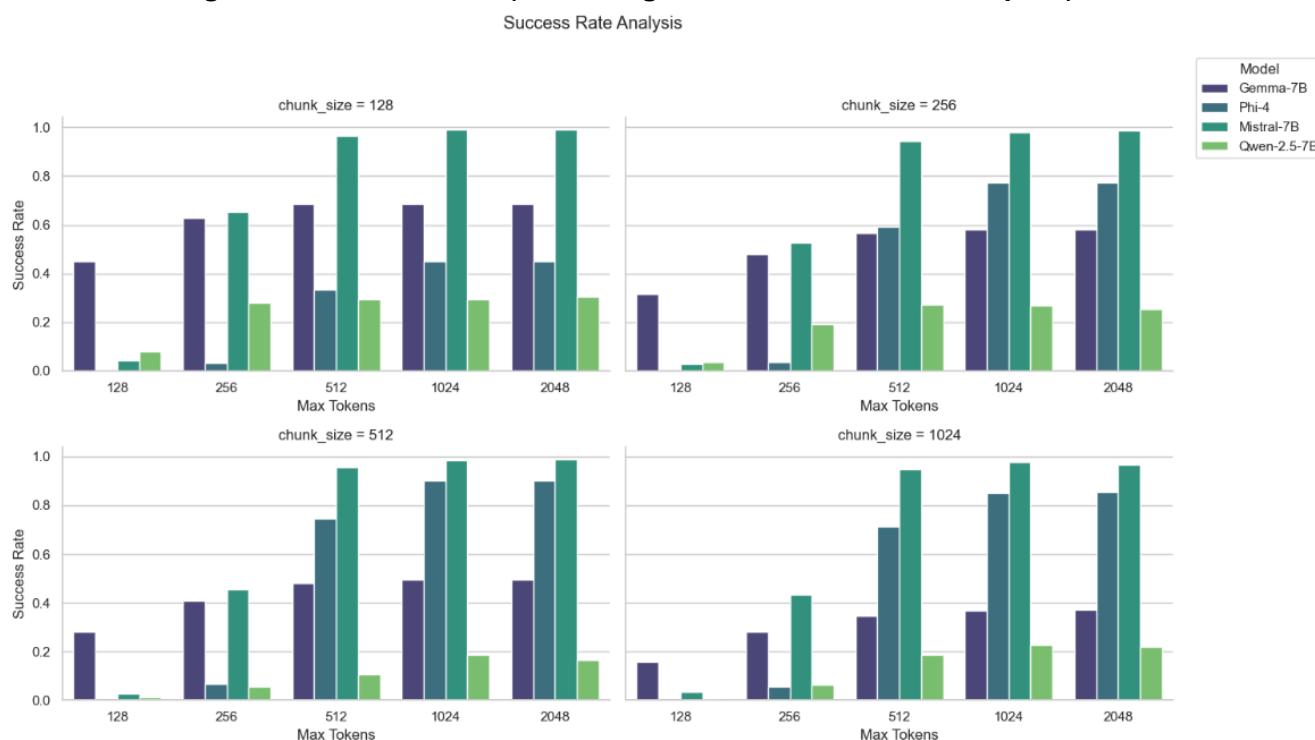


Reliability and Success Rate

For non-technical business researchers, the system must function as a "black box" where inputs reliably produce usable outputs without requiring code-level debugging. We defined "reliability" as the success rate of generating outputs that could be strictly parsed into the valid JSON format necessary for model fine-tuning.

Figure 5 shows the generation success rate (i.e., parseable JSON output) for the four models, differentiated by color, with four charts for each chunk size and groupings within each chart for max output token sizes. Overall, Mistral-7B demonstrated superior reliability, achieving the highest success rate with significantly fewer generation failures compared to other models. For example, at the 1024-token configuration, Mistral encountered only 23 failures, whereas Qwen2.5-7B suffered a critical failure rate with 1,064 invalid outputs. While Qwen2.5-7B is a capable model, its inability to strictly adhere to formatting constraints makes it ineffective for automated pipelines without extensive error-handling mechanisms or further fine-tuning of the model.

Figure 5: Success Rate (Percentage of Parseable JSON Outputs)



Quality Assessment

We evaluated the quality of the successful generations using two metrics: BERT Score (F1) for semantic accuracy and ROUGE-L for structural adherence to the reference format. The BERT F1 score measures how well the meaning of the generated Q&A matches the source context. The ROUGE-L scores offered insight into how well models maintained phrasing and format.

Figure 6 shows the BERT F1 scores for the models, differentiated by color, across four charts representing different input chunk sizes; each chart shows multiple model groupings representing different max output token sizes. The BERT F1 scores ranged from near 0.3 to above 0.6. Gemma-7B consistently underperformed the other models in semantic alignment, indicating that the question-answer pairs were less semantically aligned with the source text compared to the other models. Mistral-7B and Qwen-2.5-7B were the most consistently high scoring models, with Phi-4 showing variability in scores across different input and output token configurations.

Figure 6: BERT F1 Semantic Accuracy Score

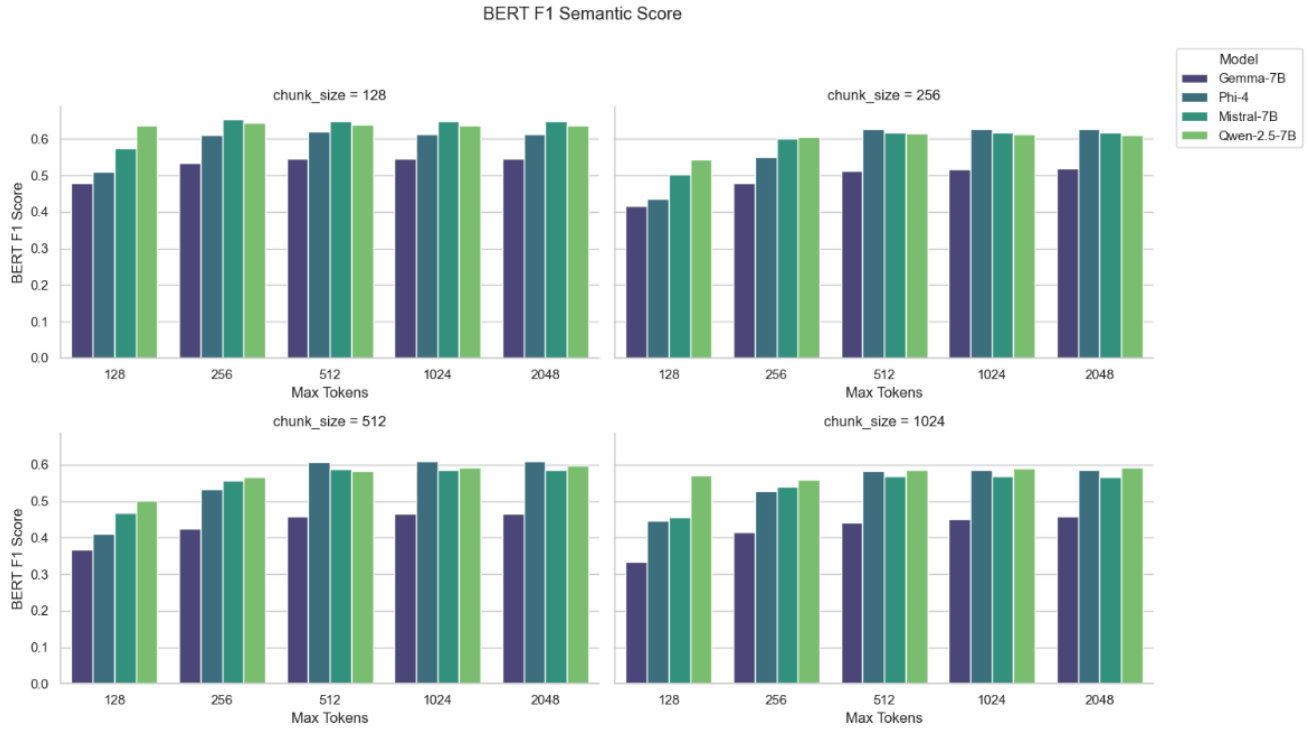


Figure 7 shows the ROUGE-L structural scores for the models, differentiated by color, across four charts representing different input chunk sizes; each chart shows multiple model groupings representing different max output token sizes. In general, ROUGE-L scores decreased as the chunk size increased. This suggests that chunk sizes should not be allowed to grow too large to maintain structural quality in the LMForge system. Again, Gemma-7B scored consistently lower than the other models on ROUGE-L. Phi-4 showed excellent structural recovery at higher max output token sizes (i.e., 512+). Interestingly, Qwen2.5 maintained a decent structure at the lowest max output token size (i.e., 128 tokens), while other models like Mistral and Phi-4 required a "minimum viable" threshold of 512 tokens to generate complete structures effectively.

Figure 7: ROUGE-L Structural Quality Score



Overall Assessment of the Decoder Models

Based on the tradeoffs of the metrics, Mistral-7B-Instruct-v0.3 was identified as the optimal question-answer dataset generation engine for the LMForge system. It offers the highest probability of success with the lowest computational cost, while maintaining quality scores that are competitive with the best models.

Discussion

This study adopts a “pure technology” design science approach for an early-stage software application designed to help researchers create custom AI models with limited knowledge of programming and AI workflows. The software artefact we are developing, Language Model Forge (LMForge), seeks to further simplify graphical AI workflow tools like n8n and LangFlow. Although n8n and LangFlow open AI model customization to many users, these tools still require users to understand details about AI workflow configuration. Using cognitive load theory, bounded rationality theory, and the technology acceptance model as kernel design theories, we suggest that a more simplified AI workflow application would require embedding common workflows into linear interaction user interface views with major parts of the AI workflow hidden from the user and automated through AI models. A first major technical hurdle in designing the artefact is how to assist non-technical users in creating custom datasets that could be used for fine-tuning models or even for retrieval augmented generation.

We proposed a two model approach to solving this problem, including an encoder-based transformer model to analyze the relevance of source data and a decoder-based transformer

model to generate question-answer fine-tuning data pairs from relevant source data. To determine the best models for this design, we examined 20 encoder models of varying sizes and 4 decoder models in the 7-billion parameter range. All models were selected to work on hardware affordable by small research labs.

Based on the results of Study 1, we found some ideal candidate models for LMForge to automate dataset creation. For the encoder model, we found that the largest of models have inference times that are too long for small-scale lab equipment. The user would be left waiting for dataset generation for far longer than is desirable. We also found that the medium models were equivalent in their accuracy, with substantially lower inference times. Although the small models exhibited better inference times, many of their accuracy scores were lower than the medium and large models. The best performing encoder models included one small model, albert-base-v1, at the 256 token chunk size; and two medium models, deberta-v3-small and distilbert, at the 256 token chunk size. Should a large chunk size (i.e., 512 tokens) be needed, a medium-sized model named roberta had strong accuracy scores and an acceptable inference time (11.96 seconds).

The results of Study 2 showed that the Mistral-7B model balanced the best inference times with good semantic accuracy and structural quality scores. Mistral-7B performed well across all metrics with a max output token size of at least 512 tokens. Similarly, a chunk size less than 1024 was desirable for the model to avoid losing structural quality in the generated outputs, with lower chunk sizes increasing structural quality.

Based on the results of the two studies, the ideal configuration of the two model design approach appears to consist of a chunk size of 256 input tokens to benefit from the strengths of the 256 token encoder model configurations, and a max output token size of 512 tokens to improve generation quality. With the 256 token chunk size, albert-base-v1, deberta-v3-small, or distilbert can be used effectively and efficiently for classifying the relevance of the source text. Then, the Mistral-7B model, outputting a max of 512 tokens, can generate fine-tuning data points from the relevant textual chunks with minimal errors.

Limitations and Future Research

The current study is one of several design science studies that will stem from the LMForge project. This early-stage “pure technology” study and later human studies will help to design better ways to democratize access to AI workflow tools for researchers with limited technical knowledge. Future studies will test the quality of the outputs of the selected configurations in this study with human users and domain experts to further assess the quality of the model outputs. Other common AI workflows will also be refined and tested in context. Further, a comparative study of LMForge against n8n and/or LangFlow will be conducted to determine if a simplified user interface does improve the ability of non-technical users to customize AI models for their intended uses.

For this initial “pure technology” design science study, further analysis is also needed. This study currently presents only the values of the metrics. We are in the process of analyzing the

experimental data using analysis of variance techniques to explore statistical significance in model differences. The results of these studies will help us better determine which models should be used as the candidate model. It may be that several models' differences are statistically insignificant, meaning they could be interchanged. Last, some design decisions were used to reduce training times to meet conference deadlines. More complete model training, such as all encoder models trained with the same number of training epochs, will be conducted as we continue toward final versions of the models and move beyond "pure technology" design science studies.

Conclusion

As AI continues to develop, non-technical users will need better access to these tools to remain competitive and gain full use of AI capabilities. LMForge and other projects like it will help to identify important design considerations for democratizing access to AI in a way that is both user friendly and of high quality.

References

- Arawjo, I., Swoopes, C., Vaithilingam, P., Wattenberg, M., & Glassman, E. L. (2024, May). Chainforge: A visual toolkit for prompt engineering and LLM hypothesis testing. In Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (pp. 1-18).
- Cao, S., Jiang, W., Wang, J., & Yang, B. (2024). From man vs. machine to man+ machine: The art and AI of stock analyses. *Journal of Financial Economics*, 160, 103910.
- Danielsson, J., Macrae, R., & Uthemann, A. (2022). Artificial intelligence and systemic risk. *Journal of Banking & Finance*, 140, 106290.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319–340.
- Elhajjar, S., Yacoub, L., & Yaacoub, H. (2023). Automation in business research: systematic literature review. *Information Systems and e-Business Management*, 21(3), 675-698.
- Khalifa, M., & Albadawy, M. (2024). Using artificial intelligence in academic writing and research: An essential productivity tool. *Computer Methods and Programs in Biomedicine Update*, 100145.
- Kim, A., Muhn, M., Nikolaev, V. V., & Tan, I. (2024). Large language models and financial reporting oversight. PCAOB 2024 conference.
- Ligo, A. K., Rand, K., Bassett, J., Galaitsi, S. E., Trump, B. D., Jayabalasingham, B., ... & Linkov, I. (2021). Comparing the emergence of technical and social sciences research in artificial intelligence. *Frontiers in Computer Science*, 3, 653235.

Liu, X. Y., Wang, G., Yang, H., & Zha, D. (2023). Fingpt: Democratizing internet-scale data for financial large language models. arXiv preprint arXiv:2307.10485.

Mariani, M. M., Machado, I., Magrelli, V., & Dwivedi, Y. K. (2023). Artificial intelligence in innovation research: A systematic review, conceptual framework, and future research directions. *Technovation*, 122, 102623.

Simon, H. A. (1955). A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1), 99–118.

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.

Sweller, J., van Merriënboer, J. J. G., & Paas, F. G. W. C. (1998). Cognitive architecture and instructional design. *Educational Psychology Review*, 10, 251–296.

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: a framework for evaluation in design science research. *European journal of information systems*, 25(1), 77-89.

Wall, J. D. (2023). Theorizing a Low Entry-Cost, Interdisciplinary Undergraduate Curriculum for Open Source Project-Based Learning.

Wieringa, R. (2014). Design science methodology for information systems and software engineering. Springer-Verlag Berlin Heidelberg.

Yang, X., Zang, S., Ren, Y., Peng, D., & Wen, Z. (2024). Evaluating Large Language Models on Financial Report Summarization: An Empirical Study. arXiv preprint arXiv:2411.06852.

Appendix A: Dataset Creation Workflow in LMForge

The models examined in this study will be integrated into the LMForge application. Screenshots of some of LMForge's pages are included in this appendix.

The Landing Page

The landing page gives users access to RAG databases, fine-tuning datasets, and models they have created with the system.

LMForge

Welcome to LMForge
This software is designed to simplify the process of creating datasets for fine-tuning language models and fine-tuning new models for specific purposes. The software is configurable to work with common AI tools, including HuggingFace, Weights and Biases, and OpenAI. If you haven't, please [configure the software here](#).

Qdrant not installed. Vector database features are disabled.

Database Chunks Collections

+ New Vector Database

Collection	Chunks	Action
No collections found.		

Datasets

+ New Dataset

Dataset	Created	Downloads	Action
OpenFinAI/FINOPT_QA_V2-split-dataset	Jan. 24, 2025, 8:28 p.m.	4	Edit
OpenFinAI/FINOPT_QA_SmolLM2_8F16-split-dataset	Jan. 27, 2025, 2:46 p.m.	1	Edit
OpenFinAI/FINOPT_QA_OPENELM-split-dataset	Jan. 27, 2025, 2:52 p.m.	6	Edit
OpenFinAI/FINOPT_QA_SMALL-train-dataset	Feb. 5, 2025, 4:39 p.m.	3	Edit
OpenFinAI/FINOPT_QA_SMALL-test-dataset	Feb. 5, 2025, 4:39 p.m.	5	Edit

Models

+ New Model

Model	Created	Downloads	Action
OpenFinAI/GPT2_FINOPT_QA	Nov. 22, 2024, 5:57 a.m.	1	Edit

Copyright 2025. All rights reserved.

Dataset Ingestion

Users can upload files or collect data sources through other means to load source data into LMForge. It is this source data that can later be selected to create a fine-tuning dataset.

LMForge

Document Processor

You can add a new document in three ways: from a URL, from a PDF, or by writing/pasting text.

Title:

Principles of Finance

Select Input Method:

Upload PDF

Upload PDF:

Browse...

PrinciplesofFinance-WEB.pdf

Select Output Format:

Text

Upload and Convert

PDF converted and saved

Uploaded/Scraped Documents

Proceed to Dataset Workflow

Converted PDF Content (TEXT)

Copyright 2025. All rights reserved

Data Source Selection

After uploading data, users can select from uploaded source data to create a new fine-tuning dataset.

LMForge

Dataset Workflow

Add new documents (ex. PDFs, websites, plain text, etc.) to include in your dataset. Once all of the desired documents are uploaded, select those you wish to include in the dataset and move to the next step.

Processed Documents

+ New Document

Delete Selected

Select	ID	Created	File	Source	Action
☒	1	Jan. 20, 2026, 1:49 a.m.	text	Principles of Finance	Delete
☒	2	Jan. 20, 2026, 1:48 a.m.	text	Legal Aspects of Corporate Management and Finance	Delete
☐	3	Jan. 20, 2026, 1:45 a.m.	reddit_subreddits_full	Reddit Business	Delete
☐	4	Jan. 20, 2026, 1:42 a.m.	text	DHS_NIST_Response	Delete
☐	5	Jan. 8, 2026, 3:44 p.m.	text	skynews	Delete
☐	6	Jan. 8, 2026, 3:42 p.m.	text	PDF Scrape Test	Delete
☐	7	Jan. 8, 2026, 3:29 p.m.	text	PDF	Delete
☐	8	Jan. 8, 2026, 3:22 p.m.	reddit_subreddits_full	Reddit_money	Delete
☐	9	Dec. 10, 2025, 2:53 a.m.	reddit_subreddits_full	Reddit_Data	Delete

Proceed with Selected

Page 1 of 1

Version 1.0.0.1

Once users select the source data they want to include in a fine-tuning dataset, they are taken to a page where with a click of a button, the source data is transformed with the two models into the appropriate fine-tuning dataset format. The data can then be saved to HuggingFace with the click of a button or downloaded in JSON or CSV format.

27